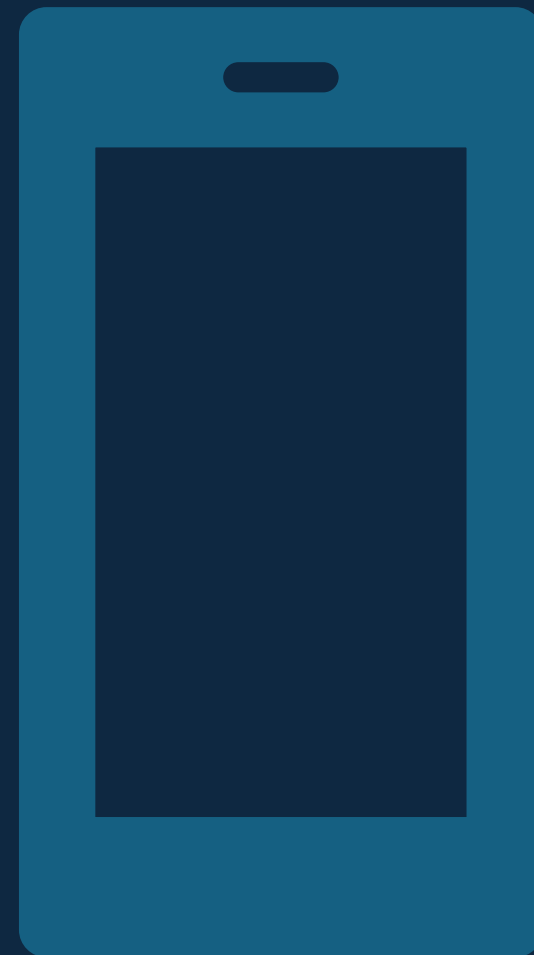


Sistema de Notificações em Tempo Real

Material
MOBILE_009





Sistema de Notificações em Tempo Real

- **Objetivo Geral:** Desenvolver uma aplicação móvel de comunicação instantânea, capaz de receber alertas em tempo real disparados por eventos no Banco de Dados, sem a necessidade de atualização manual da tela.

Competências a Desenvolver

➤ Arquitetura Realtime (WebSockets):

- Compreender a diferença entre **Polling** (perguntar ao servidor) e **Listening** (ser avisado pelo servidor) utilizando o **Supabase Realtime**.

➤ Gestão de Banco de Dados na Nuvem:

- Criar tabelas e configurar políticas de segurança (**RLS - Row Level Security**) para controlar o acesso de leitura e escrita.

➤ Notificações Mobile:

- Implementar a biblioteca **Expo Notifications** para gerenciar permissões, sons e a exibição de pop-ups visuais no sistema operacional (Android/iOS).

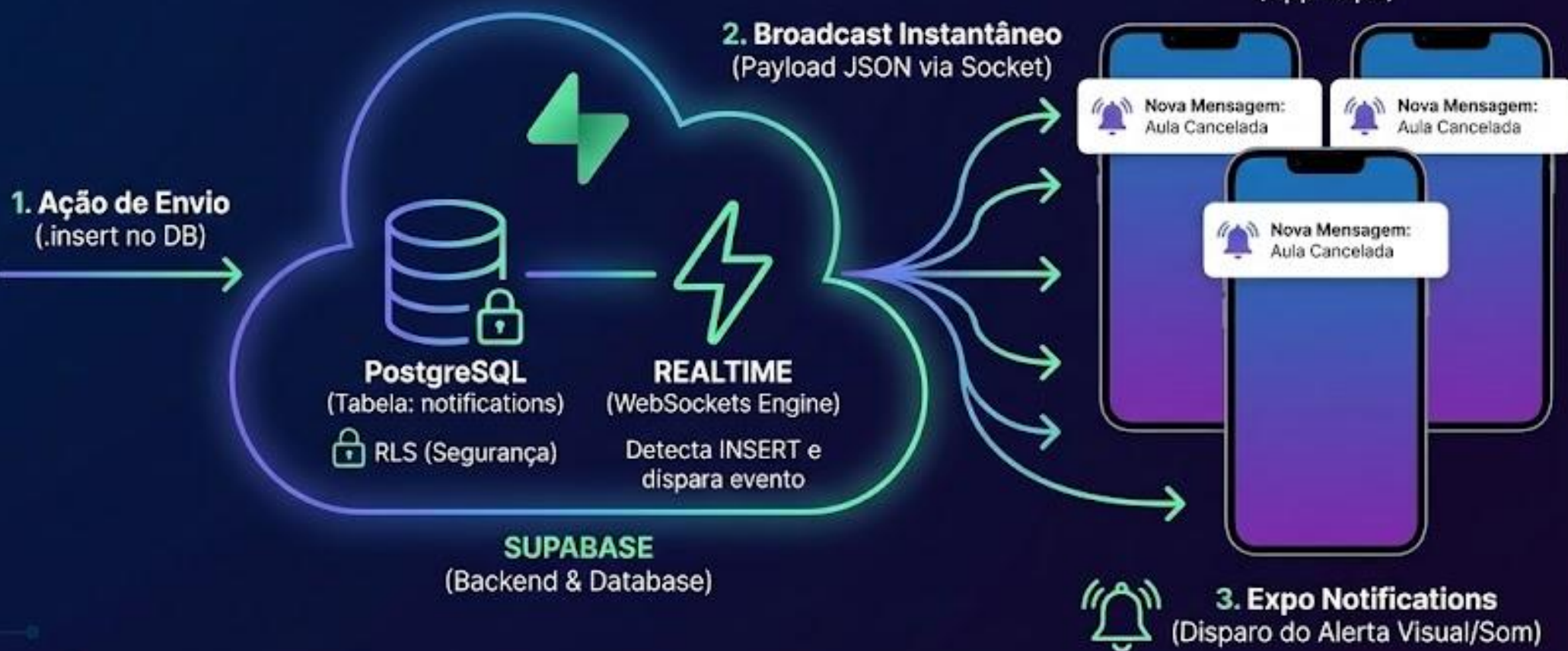
➤ Integração Backend-Frontend:

- Conectar uma aplicação React Native a um serviço de **Backend as a Service (BaaS)** para sincronização de dados.

ENGCOMP ALERT: Arquitetura de Notificações em Tempo Real (Expo + Supabase)

COORDENADOR
(App Expo)

ALUNOS
(App Expo)



React Native
(UI)



Expo
(Framework & Notificações)



Supabase
(DB & Auth)



WebSockets
(Comunicação)



Alerta em Tempo Real

➤ 1) O Objetivo da Aula

- Criar um sistema de avisos para a Engenharia de Computação.

➤ Cenário:

- O Coordenador envia um aviso ("Mudança de sala", "Nota lançada").

➤ Resultado:

- Todos os alunos com o app aberto recebem o alerta instantaneamente no celular.

- **Diferencial:** Usaremos tecnologia **Realtime (WebSockets)** em vez de *Push Notifications* tradicionais, contornando limitações do Expo Go e garantindo entrega instantânea.

Tecnologias Usadas

- **Expo (React Native):** Para criar o aplicativo mobile que roda em Android e iOS.
- **Supabase Database (PostgreSQL):** Para armazenar o histórico de mensagens com segurança.
- **Supabase Realtime:** Um "radar" que escuta o banco de dados. Assim que uma mensagem é salva, ele avisa o aplicativo.
- **Expo Notifications:** Para exibir o alerta visual (pop-up) no topo da tela do aluno.

Configuração do Backend (Supabase)

- Antes do código, precisamos preparar a "Nuvem".
- Criar a Tabela:
 - 1. Acesse seu projeto no **Supabase**.
 - 2. Vá em **Table Editor > New Table**.
 - 3. Nome: **notifications**.
 - 4. IMPORTANTE: Marque a opção **Enable Realtime**. (**Sem isso, o app não atualiza sozinho**).

Veja Figura no Próximo Slide

Name

notifications

Description

Optional



Enable Row Level Security (RLS) RECOMMENDED

Restrict access to your table by enabling RLS and writing Postgres policies.



Policies are required to query data

You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an empty array of results.



Documentation



Enable Realtime

Broadcast changes on this table to authorized subscribers.

Columns

About data types

Name ?

Type

Default Value ?

Primary

Cancel

Save Ctrl ⌘

Campos da Tabela

➤ Colunas:

- id (int8, primary key, auto increment)
- title (text)
- body (text)
- created_at (now)

Clique em **Save**.

Columns

[About data types](#)

Name ?	Type	Default Value ?	Primary
id	# int8	NULL	☒
created_at	📅 timestamptz	now()	☐
title	T text	NULL	☐
body	T text	NULL	☐

Add column

Nota importante

- **O RLS ativado (é uma boa prática).** Agora você precisa criar uma "regra" dizendo que todo mundo pode mexer nessa tabela:
- Vá em **Authentication** (ícone do cadeado no menu lateral) > **Policies**.
- Encontre a tabela books.
- Clique em **Create Policy**. Para Exibir a caixa de diálogo "Create a new Row Level Security policy"
- Defina como a próxima figura. Policy Name "Enabled read/write for all users". Deixe como o quadro a seguir:

Create a new Row Level Security Policy

Policy Name: Enabled read/write for all users

Table on clause: public.notifications

Policy Behavior as clause: Permissive

Policy Command for clause: ☐ SELECT ☐ INSERT ☐ UPDATE ☐ DELETE ☒ ALL

Target Roles to clause: Defaults to all (public) roles if none selected

USE OPTIONS ABOVE TO EDIT

```
1 alter policy "Enabled read/write for all users"
2 on "public"."notifications"
5 to public
6 using (
7   true
8 ) with check (
9   true
10 );
```

☒ Use check expression

Cancel Save policy

Preparação do App - Frontend

► No terminal do projeto

1. Criar o projeto

npx create-expo-app EngCompAlert -t blank

cd EngCompAlert

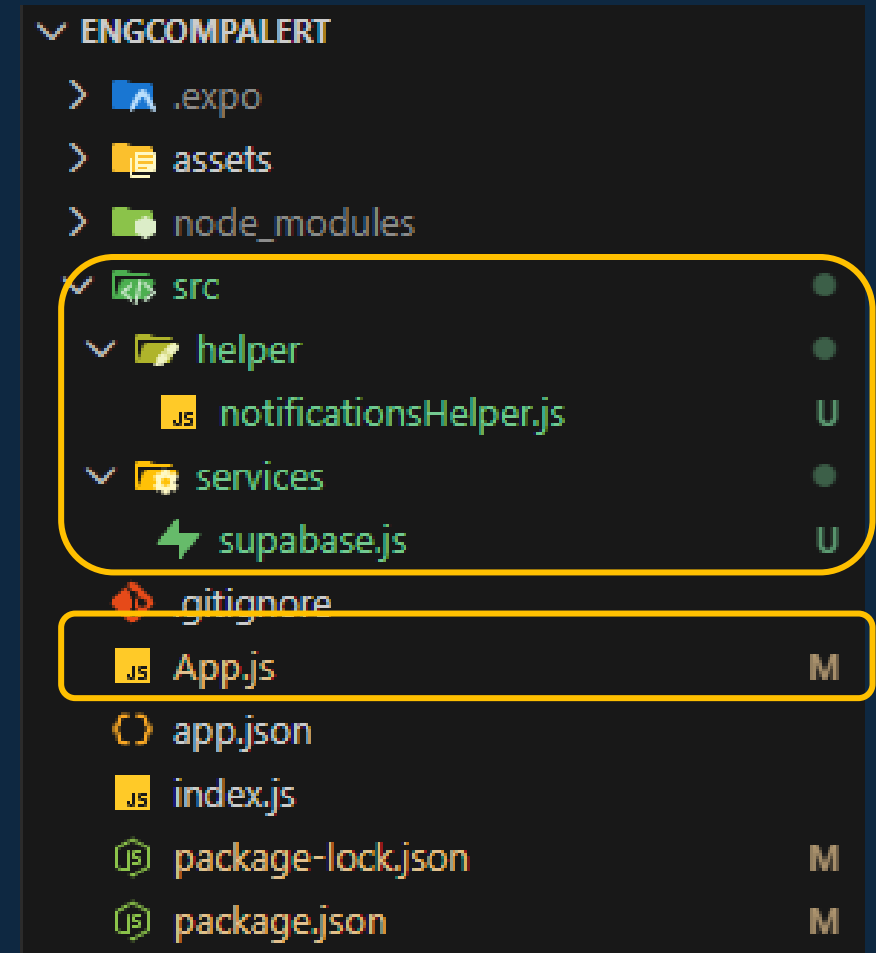
2. Instalar as bibliotecas de Notificação e Banco de Dados

npx expo install expo-notifications expo-device

npm install @supabase/supabase-js react-native-url-polyfill

Crie 03 arquivos principais

- 1. src/services/supabase.js (Conexão)
- 2. src/helper/notificationsHelper.js
- 3. App.js



Passo 1 – Configurar conexão

Arquivo: src/services/supabase.js

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';

// Substitua pelas suas chaves do painel do Supabase (Project Settings > API)
const supabaseUrl = 'SUA_URL_DO_SUPABASE';
const supabaseKey = 'SUA_CHAVE_ANON_PUBLICA';

export const supabase = createClient(supabaseUrl, supabaseKey);
```

API Keys

Configure API keys to securely control access to your project

Publishable and secret API keys Legacy anon, service_role API keys

anon public

eyJhbGciOiJIUzI1NiIsInR5cCI6I

This key is safe to use in a browser if you
Prefer using Publishable API keys instead

service_role **secret**

**** * * * *

This key has the ability to bypass Row Level Security
immediately. Prefer using Secret API keys

Na dúvida, assista:

<https://www.youtube.com/watch?v=EAs3PRquQtw>

“

**Passo 2: O Gerenciador de
Alertas (notificationsHelper.js):**

**Este arquivo cuida da
permissão e da exibição da
notificação.**

”

Próximos Slides. Explicação do Professor.



```
// src/helper/notificationsHelper.js
import * as Notifications from 'expo-notifications';

// Configuração: Permitir som e alerta mesmo com app aberto
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});

// Função para pedir permissão (Necessário para Local Notification também)
export async function registerForLocalNotificationsAsync() {
  const { status: existingStatus } = await Notifications.getPermissionsAsync();
  let finalStatus = existingStatus;

  if (existingStatus !== 'granted') {
    const { status } = await Notifications.requestPermissionsAsync();
    finalStatus = status;
  }
}
```




```
if (finalStatus !== 'granted') {  
  alert('Sem permissão para notificações!');  
  return false;  
}  
return true;  
}  
  
// Função que dispara o alerta no próprio celular  
export async function triggerLocalNotification(title, body) {  
  await Notifications.scheduleNotificationAsync({  
    content: {  
      title: title,  
      body: body,  
      sound: true,  
    },  
    trigger: null, // null = Disparar agora mesmo  
  });  
}
```



Código compartilhado
Link abaixo:

<https://drive.google.com/file/d/15IJQrfka0uDztiEQmX6CgvVSHI5g6LFF/view?usp=sharing>

“

Passo 3: Juntar tudo no App.js

”

Próximos Slides. Explicação do Professor.



```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View, TextInput, Button, Alert, ActivityIndicator, ScrollView } from 'react-native';
import { supabase } from '../src/services/supabase';
import { registerForLocalNotificationsAsync, triggerLocalNotification } from '../src/helper/notificationsHelper';

export default function App() {
  const [title, setTitle] = useState('');
  const [message, setMessage] = useState('');
  const [loading, setLoading] = useState(false);
  const [permissionGranted, setPermissionGranted] = useState(false);

  // 1. INICIALIZAÇÃO E ESCUTA (REALTIME)
  useEffect(() => {
    // A) Pede permissão para mostrar notificações
    registerForLocalNotificationsAsync().then(granted => {
      setPermissionGranted(granted);
    });

    // B) Liga o "Radar" do Supabase (Realtime)
    const subscription = supabase
      .channel('public:notifications') // Canal público
      .on(
        'postgres_changes',
        { event: 'INSERT', schema: 'public', table: 'notifications' },
        (payload) => {
          // C) MÁGICA: Quando chegar um dado novo, dispara notificação
          console.log("Nova mensagem recebida!", payload.new);
          triggerLocalNotification(payload.new.title, payload.new.body);
        }
      )
      .subscribe();

    // Limpeza ao fechar o app
    return () => supabase.removeChannel(subscription);
  }, []);
```



```
// 2. FUNÇÃO DO ADMIN (Enviar)
const sendNotification = async () => {
  if (!title.trim() || !message.trim()) return Alert.alert("Preencha tudo");

  setLoading(true);

  // Em vez de chamar API da Expo, apenas SALVAMOS no banco.
  // O Realtime fará o resto.
  const { error } = await supabase
    .from('notifications')
    .insert([ { title: title, body: message } ]);

  setLoading(false);

  if (error) {
    Alert.alert("Erro", error.message);
  } else {
    Alert.alert("Sucesso", "Mensagem enviada para todos conectados!");
    setTitle('');
    setMessage('');
  }
};

return (
  <View style={styles.container}>
    <ScrollView contentContainerStyle={styles.scroll}>
      <Text style={styles.header}>🔔 EngComp Alert (Realtime)</Text>

      <View style={styles.statusContainer}>
        <Text style={styles.label}>Status da Conexão:</Text>
        <Text style={{ color: permissionGranted ? 'green' : 'orange', fontWeight: 'bold' }}>
          {permissionGranted ? "🟢 Escutando Avisos..." : "🔴 Sem Permissão"}
        </Text>
      </View>
    </ScrollView>
  </View>
)
```



```
<View style={styles.divider} />

<Text style={styles.subHeader}>Área do Coordenador</Text>
<TextInput
  style={styles.input}
  placeholder="Título (ex: Nota Liberada)"
  value={title} onChangeText={setTitle}
/>
<TextInput
  style={[styles.input, styles.textArea]}
  placeholder="Mensagem (ex: Confira no SIGA)"
  value={message} onChangeText={setMessage}
  multiline
/>

{loading ? (
  <ActivityIndicator size="large" color="#007AFF" />
) : (
  <Button title="DISPARAR ALERTA" onPress={sendNotification} />
)}
</ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5', paddingTop: 50 },
  scroll: { padding: 20 },
  header: { fontSize: 24, fontWeight: 'bold', textAlign: 'center', marginBottom: 20, color: '#333' },
  statusContainer: { backgroundColor: '#fff', padding: 15, borderRadius: 10, alignItems: 'center' },
  label: { fontSize: 14, color: '#666' },
  divider: { height: 1, backgroundColor: '#ccc', marginVertical: 20 },
  subHeader: { fontSize: 20, fontWeight: 'bold', color: '#007AFF', marginBottom: 10 },
  input: { backgroundColor: '#fff', padding: 15, borderRadius: 8, marginBottom: 15, fontSize: 16 },
  textArea: { height: 80, textAlignVertical: 'top' },
});
```



Código compartilhado
Link abaixo:

<https://drive.google.com/file/d/1wD-Q2lbcY80xYuZWVdiNtEzCiDhTD5YR/view?usp=sharing>

Testando

1.Rodar: Alunos rodem `npx expo start --tunnel`

2.Conectar: Cada aluno escaneia o QR Code com seu próprio celular (App Expo Go).

3.Permissão: O app vai pedir permissão de notificação. Eles devem aceitar.

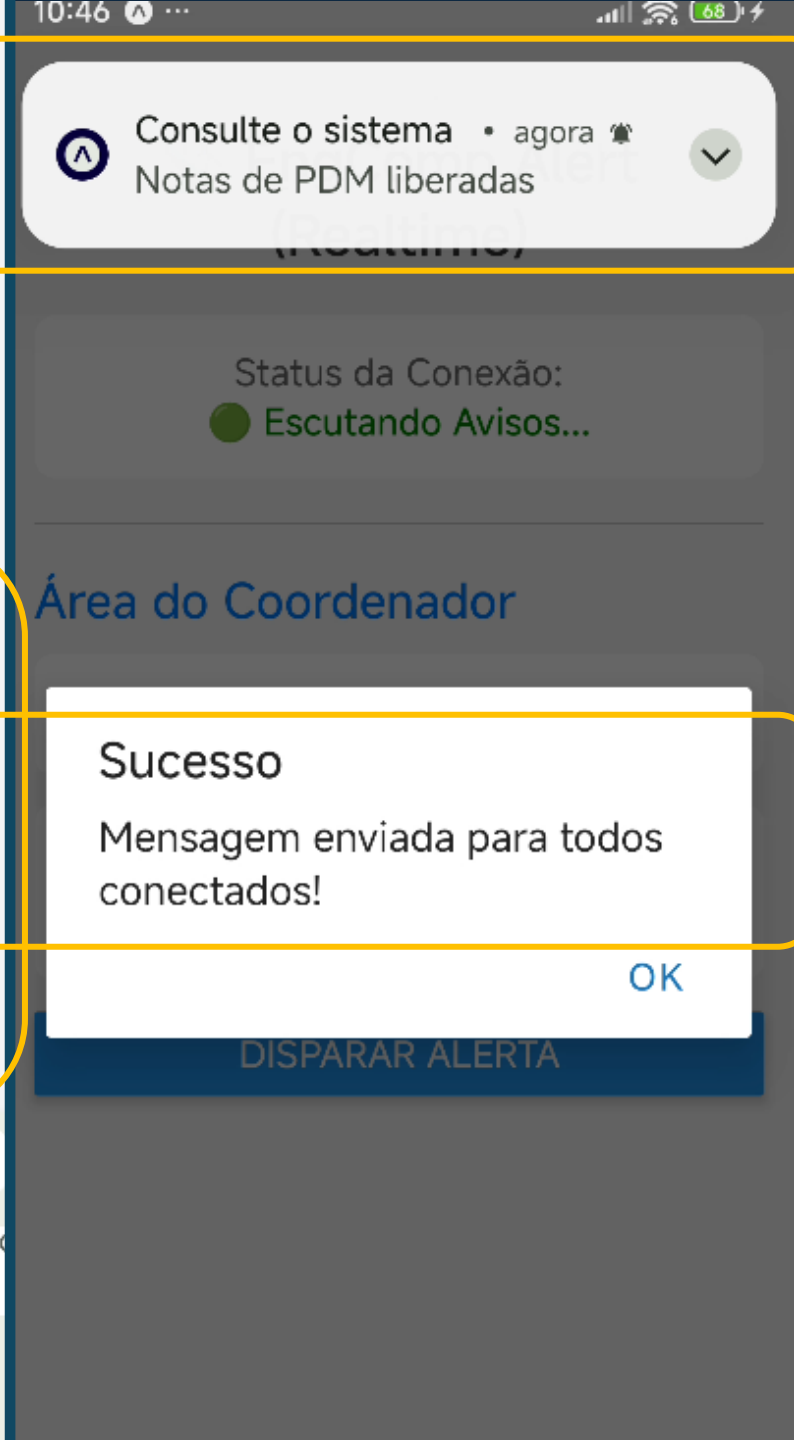
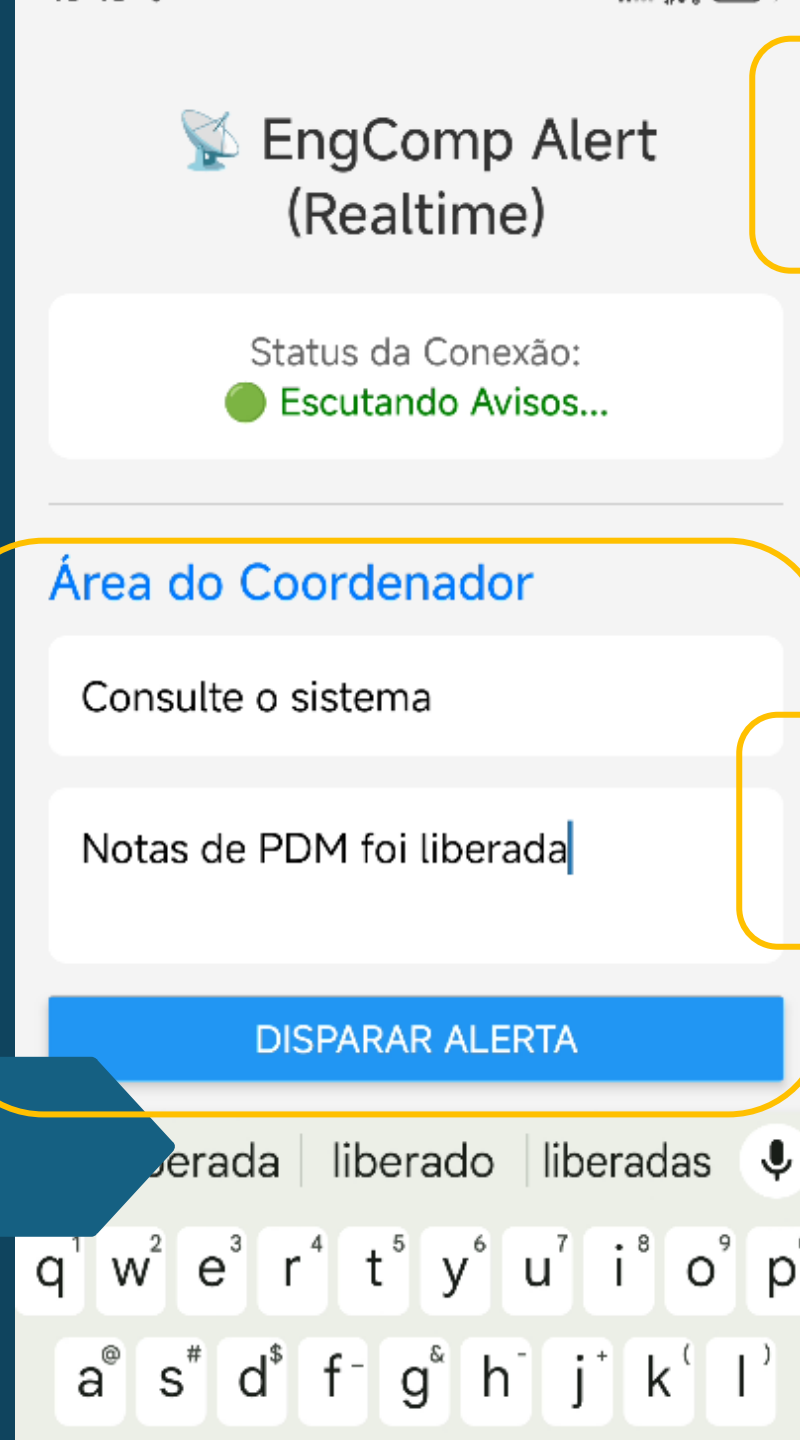
4.Ação:

1. Professor digita uma mensagem.

5.Resultado: Todos os celulares da sala vibrarão e mostrarão a notificação ao mesmo tempo, reagindo à inserção no banco de dados.

Há limitações para **push do servidor a partir da versão 53**. A mensagem de erro, que não impede o funcionamento. Uma opção seria criar um Development Build (para push remotos). [eas build --platform android --profile development]

Resultados



Resumo da Aula

- O que fizemos aqui? Em vez de enviar uma mensagem direta de um celular para outro, nós enviamos a mensagem para o **Banco de Dados (Supabase)**.
- Como habilitamos o **Realtime** e as **Políticas de Segurança (RLS)**, o banco de dados avisou a todos os aplicativos conectados que houve uma mudança. O aplicativo, ao saber disso, disparou o alerta localmente. É assim que funcionam chats e feeds ao vivo."

Fim



Prof. Maromo